```
        senderId, Attribute.UNPROTECTED);    #C
message.SetContent(    #D
    new byte[] { 0x01, (byte) 0xb3, 0x74, 0x76, 0x31});    #D

var associatedData = CBORObject.NewArray();    #E
associatedData.Add(1);    #E
associatedData.Add(algorithm.AsCBOR());    #E
associatedData.Add(senderId);    #E
associatedData.Add(partialIv);    #E
associatedData.Add(new byte[0]);    #E
message.setExternal(associatedData.EncodeToBytes());    #E

Security.addProvider(new BouncyCastleProvider());    #F
message.encrypt(senderKey.getEncoded());    #F
```

#A Generate the nonce and encode the Partial IV
#B Configure the algorithm and nonce
#C Set the Partial IV and Sender ID as unprotected headers
#D Set the content field to the plaintext to encrypt
#E Encode the external associated data
#F Ensure Bouncy Castle is loaded for AES-CCM support then encrypt the message

The encrypted message is then encoded into the application protocol, such as CoAP or HTTP and sent to the recipient. Details of this encoding are given in section 6 of the OSCORE specification. The recipient can recreate the nonce from its own recipient security context, together with the Partial IV and Sender ID encoded into the message.

The recipient is responsible for checking that the Partial IV has not been seen before to prevent replay attacks. When OSCORE is transmitted over a reliable protocol such as HTTP then this can be achieved by keeping track of the last Partial